

```
In [1]: import numpy as np
import matplotlib
matplotlib.use('module://matplotlib_inline.backend_inline')
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
import rustworkx as rx
from rustworkx.visualization import mpl_draw
from qiskit import QuantumCircuit, transpile
from qiskit.visualization import plot_histogram
from qiskit_ibm_runtime.fake_provider import FakeFez
from qiskit_aer import AerSimulator

# =====
# BACKEND SELECTION FLAG
# Set USE_REAL_HARDWARE = True to run on actual IBM quantum hardware
# Set USE_REAL_HARDWARE = False to use the FakeFez simulator
# =====
USE_REAL_HARDWARE = True

if USE_REAL_HARDWARE:
    # Load real IBM Quantum backend
    import json
    import sys
    sys.path.append('.')
    from qiskit_ibm_runtime import QiskitRuntimeService

    # Load credentials from secrets
    with open('../secrets/keys.json') as f:
        keys = json.load(f)

    service = QiskitRuntimeService(
        channel='ibm_quantum_platform',
        token=keys["qiskit-api-key"],
        instance=keys["qiskit-crn-instance"]
    )

    # Get ibm_fez backend (156 qubits, same topology as FakeFez)
    backend = service.backend('ibm_fez')
    coupling_map = backend.coupling_map
    print(f"Loaded REAL backend: {backend.name} with {backend.num_qubits} qubits")
    print(f"Backend status: {backend.status().status_msg}")
else:
    # Use FakeFez simulator
    backend = FakeFez()
    coupling_map = backend.coupling_map
    print(f"Loaded SIMULATOR backend: {backend.name} with {backend.num_qubits} qubits")
```

qiskit_runtime_service._discover_account:WARNING:2026-01-09 18:04:15,339: Loading account with the given token. A saved account will not be used.

Loaded REAL backend: ibm_fez with 156 qubits
Backend status: active

```
In [2]: # Explore topology for new qubit configurations
# Source: [81, 76, 61]
# Small Edge Dest: [63, 51, 43]
# Large Edge Dest: [65, 77, 85]

edges = list(coupling_map.get_edges())
adj = {i: set() for i in range(backend.num_qubits)}
for e in edges:
    adj[e[0]].add(e[1])
    adj[e[1]].add(e[0])

print("=== Topology Analysis for New Configurations ===\n")

# Check source cluster connectivity
print("SOURCE CLUSTER [81, 76, 61] (top, data, bottom):")
for q in [81, 76, 61]:
    print(f"  {q}: neighbors = {sorted(adj[q])}")
print(f"  76-81 connected? {81 in adj[76]}")
print(f"  76-61 connected? {61 in adj[76]}")

print("\nSMALL EDGE DEST [63, 51, 43] (top, data, bottom):")
for q in [63, 51, 43]:
    print(f"  {q}: neighbors = {sorted(adj[q])}")
print(f"  51-63 connected? {63 in adj[51]}")
print(f"  51-43 connected? {43 in adj[51]}")

print("\nLARGE EDGE DEST [65, 77, 85] (top, data, bottom):")
for q in [65, 77, 85]:
    print(f"  {q}: neighbors = {sorted(adj[q])}")
print(f"  77-65 connected? {65 in adj[77]}")
print(f"  77-85 connected? {85 in adj[77]}")

# Check if 56 could be the data qubit for small edge (connecting 63 and 43)
print("\n=== Checking alternative for Small Edge ===")
print(f"56: neighbors = {sorted(adj[56])}")
print(f"  56-63 connected? {63 in adj[56]}")
print(f"  56-43 connected? {43 in adj[56]}")

# Find paths using BFS
from collections import deque
```

```
def bfs_path(adj, start, end):
    if start == end:
        return [start]
    visited = {start}
    queue = deque([(start, [start])])
    while queue:
        node, path = queue.popleft()
        for neighbor in adj[node]:
            if neighbor == end:
                return path + [neighbor]
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))
    return None

# Find travel paths (from bottom qubit to bottom qubit)
print("\n=== TRAVEL PATHS ===")

# Small Edge: 61 → 43
path_small = bfs_path(adj, 61, 43)
print(f"\nSmall Edge (61 → 43): {' → '.join(map(str, path_small))}")
print(f"  Travel path (middle): {path_small[1:-1]}")
print(f"  Hops: {len(path_small)-1}")

# Large Edge: 61 → 85
path_large = bfs_path(adj, 61, 85)
print(f"\nLarge Edge (61 → 85): {' → '.join(map(str, path_large))}")
print(f"  Travel path (middle): {path_large[1:-1]}")
print(f"  Hops: {len(path_large)-1}")
```

=== Topology Analysis for New Configurations ===

SOURCE CLUSTER [81, 76, 61] (top, data, bottom):

```
81: neighbors = [76, 80, 82]
76: neighbors = [61, 81]
61: neighbors = [60, 62, 76]
76-81 connected? True
76-61 connected? True
```

SMALL EDGE DEST [63, 51, 43] (top, data, bottom):

```
63: neighbors = [56, 62, 64]
51: neighbors = [50, 52, 58]
43: neighbors = [42, 44, 56]
51-63 connected? False
51-43 connected? False
```

LARGE EDGE DEST [65, 77, 85] (top, data, bottom):

```
65: neighbors = [64, 66, 77]
77: neighbors = [65, 85]
85: neighbors = [77, 84, 86]
77-65 connected? True
77-85 connected? True
```

=== Checking alternative for Small Edge ===

```
56: neighbors = [43, 63]
56-63 connected? True
56-43 connected? True
```

=== TRAVEL PATHS ===

Small Edge (61 → 43): 61 → 62 → 63 → 56 → 43

```
Travel path (middle): [62, 63, 56]
Hops: 4
```

Large Edge (61 → 85): 61 → 62 → 63 → 64 → 65 → 77 → 85

```
Travel path (middle): [62, 63, 64, 65, 77]
Hops: 6
```

1. Define Transport Configurations

We define both Small Edge and Large Edge configurations using the qubit assignments discovered from the topology explorer.

```
In [3]: # =====
# TRANSPORT CONFIGURATIONS FOR FakeFez
# =====
# Source cluster: [81, 76, 61] (top, data, bottom)
# - 76 connects to both 81 and 61 ✓

# --- SMALL EDGE CONFIGURATION ---
# Note: Original request was [63, 51, 43] but 51 is NOT connected to 63 or 43
# Using [63, 56, 43] instead since 56 connects to both 63 and 43
# Path: 61 → 62 → 63 → 56 → 43 (4 hops)
# Total SWAPS: 4 + 5 + 6 = 15
config_small = {
    'name': 'Small Edge',
    'source_qubits': [81, 76, 61],      # [top, data, bottom]
    'dest_qubits': [63, 56, 43],       # [top, data, bottom] - 56 is the data qubit
    'ancilla_qubits': [64, 42],        # [top_ancilla, bottom_ancilla]
    'travel_path': [62, 63, 56],       # Intermediate qubits (61-62-63-56-43)
}
```

```
# --- LARGE EDGE CONFIGURATION ---
# Path: 61 → 62 → 63 → 64 → 65 → 77 → 85 (6 hops)
# Total SWAPs: 6 + 7 + 8 = 21
config_large = {
    'name': 'Large Edge',
    'source_qubits': [81, 76, 61],      # [top, data, bottom]
    'dest_qubits': [65, 77, 85],       # [top', data', bottom']
    'ancilla_qubits': [66, 86],        # [top_ancilla, bottom_ancilla]
    'travel_path': [62, 63, 64, 65, 77], # Intermediate qubits (61-62-63-64-65-77-85)
}

# Store both configurations
configs = {'small': config_small, 'large': config_large}

print("=== FakeFez Transport Configurations ===")
for key, cfg in configs.items():
    src = cfg['source_qubits']
    dst = cfg['dest_qubits']
    travel = cfg['travel_path']
    n = len(travel) + 1 # hops
    total_swaps = n + (n+1) + (n+2)

    print(f"\n{cfg['name']}:")
    print(f" Source: {src} (top={src[0]}, data={src[1]}, bottom={src[2]})")
    print(f" Destination: {dst} (top={dst[0]}, data={dst[1]}, bottom={dst[2]})")
    print(f" Ancillas: {cfg['ancilla_qubits']}")
    print(f" Travel path: {travel}")
    print(f" Path length: {n} hops")
    print(f" Total SWAPs: {total_swaps} ({n}+{n+1}+{n+2})")
```

=== FakeFez Transport Configurations ===

Small Edge:

Source: [81, 76, 61] (top=81, data=76, bottom=61)
 Destination: [63, 56, 43] (top=63, data=56, bottom=43)
 Ancillas: [64, 42]
 Travel path: [62, 63, 56]
 Path length: 4 hops
 Total SWAPs: 15 (4+5+6)

Large Edge:

Source: [81, 76, 61] (top=81, data=76, bottom=61)
 Destination: [65, 77, 85] (top=65, data=77, bottom=85)
 Ancillas: [66, 86]
 Travel path: [62, 63, 64, 65, 77]
 Path length: 6 hops
 Total SWAPs: 21 (6+7+8)

2. Verify Topology & Connectivity

Before building circuits, verify that all required connections exist on FakeFez.

```
In [4]: def check_connection(coupling_map, u, v):
        """Returns True if physical wire exists between u and v."""
        return u in coupling_map.neighbors(v) or v in coupling_map.neighbors(u)

def verify_configuration(coupling_map, config):
    """Verify all required connections for a transport configuration."""
    print(f"\n=== Verifying {config['name']} ===")

    src_top, src_data, src_bot = config['source_qubits']
    dst_top, dst_data, dst_bot = config['dest_qubits']
    anc_top, anc_bot = config['ancilla_qubits']
    travel_path = config['travel_path']

    all_valid = True

    # Source cluster
    src_valid = check_connection(coupling_map, src_data, src_top) and check_connection(coupling_map, src_data, src_bot)
    print(f"Source ({src_top}-{src_data}-{src_bot}): {'✓' if src_valid else 'x'}")
    all_valid &= src_valid

    # Destination cluster
    dst_valid = check_connection(coupling_map, dst_data, dst_top) and check_connection(coupling_map, dst_data, dst_bot)
    print(f"Destination ({dst_top}-{dst_data}-{dst_bot}): {'✓' if dst_valid else 'x'}")
    all_valid &= dst_valid

    # Travel path
    full_path = [src_bot] + travel_path + [dst_bot]
    path_valid = True
    for i in range(len(full_path) - 1):
        if not check_connection(coupling_map, full_path[i], full_path[i+1]):
            print(f"x Travel path broken: {full_path[i]}-{full_path[i+1]}")
            path_valid = False
    print(f"Travel path ({' → '.join(map(str, full_path))}): {'✓' if path_valid else 'x'}")
    all_valid &= path_valid

    # Ancillas (just need to connect to at least one dest qubit for SWAP-assisted)
    anc_top_valid = check_connection(coupling_map, anc_top, dst_top)
    anc_bot_valid = check_connection(coupling_map, anc_bot, dst_bot)
    print(f"Top ancilla {anc_top} → {dst_top}: {'✓' if anc_top_valid else 'x'}")
    print(f"Bottom ancilla {anc_bot} → {dst_bot}: {'✓' if anc_bot_valid else 'x'}")
```

```

    all_valid &= anc_top_valid and anc_bot_valid

    print(f"\nOverall: {'✓ VALID' if all_valid else '✗ INVALID'}")
    return all_valid

# Verify both configurations
for cfg in configs.values():
    verify_configuration(coupling_map, cfg)

=== Verifying Small Edge ===
Source (81-76-61): ✓
Destination (63-56-43): ✓
Travel path (61 → 62 → 63 → 56 → 43): ✓
Top ancilla 64 → 63: ✓
Bottom ancilla 42 → 43: ✓

Overall: ✓ VALID

=== Verifying Large Edge ===
Source (81-76-61): ✓
Destination (65-77-85): ✓
Travel path (61 → 62 → 63 → 64 → 65 → 77 → 85): ✓
Top ancilla 66 → 65: ✓
Bottom ancilla 86 → 85: ✓

Overall: ✓ VALID

```

3. Visualize Transport Paths

Let's visualize both transport paths side-by-side on the FakeFz topology.

```

In [5]: # Build Heavy Hex grid layout for FakeFz (similar to FakeBrisbane visualization)
# FakeFz has 156 qubits in a Heavy Hex topology

edges = list(coupling_map.get_edges())

# Build adjacency list
adj = {i: set() for i in range(backend.num_qubits)}
for e in edges:
    adj[e[0]].add(e[1])
    adj[e[1]].add(e[0])

# FakeFz topology analysis:
# Row 0: 0-19 (horizontal chain) with vertical connections at 3-16, 7-17, 11-18, 15-19
# The vertical connections go to qubits 16-19, which connect to row 1 (20-39)
# This is a "Heavy Hex" but numbered differently than Brisbane

# Build positions based on actual connectivity
pos = {}

# Analyze connections to understand the layout
print(f"FakeFz: {backend.num_qubits} qubits, {len(edges)} edges")

# Find vertical connectors from row 0 (0-15)
print("\nVertical connections from row 0:")
for q in range(20):
    vertical = [n for n in adj[q] if n >= 16]
    if vertical:
        print(f"  {q} → {vertical}")

# Based on the connectivity pattern, FakeFz appears to have:
# - Main rows of ~20 qubits horizontally connected
# - Flag qubits connecting rows vertically

# Let's map out the structure properly
# Row 0 (bottom): qubits 0-19
for i in range(20):
    pos[i] = (i, 0)

# Row 1: qubits 20-39
for i in range(20):
    pos[20 + i] = (i, 2)

# Row 2: qubits 40-59
for i in range(20):
    pos[40 + i] = (i, 4)

# Row 3: qubits 60-79
for i in range(20):
    pos[60 + i] = (i, 6)

# Row 4: qubits 80-99
for i in range(20):
    pos[80 + i] = (i, 8)

# Row 5: qubits 100-119
for i in range(20):
    pos[100 + i] = (i, 10)

# Row 6: qubits 120-139
for i in range(20):
    pos[120 + i] = (i, 12)

```

```

# Row 7: qubits 140-155 (last 16 qubits)
for i in range(16):
    pos[140 + i] = (i + 2, 14)

# Now adjust positions based on connections to make it look like Heavy Hex
# The flag qubits (16-19 from row 0) should be positioned between rows
# Actually, looking at connections: 3-16, 7-17, 11-18, 15-19
# These are NOT flag qubits - they're part of row 0

# Let me check connections more carefully
print("\nChecking connections for qubits 16-19:")
for q in [16, 17, 18, 19]:
    print(f"  {q}: {sorted(adj[q])}")

# It looks like 16,17,18,19 connect to both row 0 and row 1
# So they ARE flag qubits! Let's reposition them

# Reposition flag qubits between rows
# Row 0 main: 0-15
# Flag 0-1: 16, 17, 18, 19 (at y=1)
# Row 1 main: 20-35
# etc.

# Actually, let's use a more accurate Heavy Hex layout:
# The qubits 16-19 appear to be flag qubits connecting rows 0 and 1

# Reposition to match Heavy Hex pattern (like Brisbane image)
# Main row 0: 0-15 at y=0
for i in range(16):
    pos[i] = (i, 0)

# Flag qubits 16-19 at y=1 (connecting row 0 to row 1)
# 16 connects to 3 and 23, 17 connects to 7 and 27, etc.
pos[16] = (3, 1)
pos[17] = (7, 1)
pos[18] = (11, 1)
pos[19] = (15, 1)

# Main row 1: 20-35 at y=2
for i in range(16):
    pos[20 + i] = (i, 2)

# Flag qubits 36-39 at y=3
pos[36] = (1, 3)
pos[37] = (5, 3)
pos[38] = (9, 3)
pos[39] = (13, 3)

# Main row 2: 40-55 at y=4
for i in range(16):
    pos[40 + i] = (i, 4)

# Flag qubits 56-59 at y=5
pos[56] = (3, 5)
pos[57] = (7, 5)
pos[58] = (11, 5)
pos[59] = (15, 5)

# Main row 3: 60-75 at y=6
for i in range(16):
    pos[60 + i] = (i, 6)

# Flag qubits 76-79 at y=7
pos[76] = (1, 7)
pos[77] = (5, 7)
pos[78] = (9, 7)
pos[79] = (13, 7)

# Main row 4: 80-95 at y=8
for i in range(16):
    pos[80 + i] = (i, 8)

# Flag qubits 96-99 at y=9
pos[96] = (3, 9)
pos[97] = (7, 9)
pos[98] = (11, 9)
pos[99] = (15, 9)

# Main row 5: 100-115 at y=10
for i in range(16):
    pos[100 + i] = (i, 10)

# Flag qubits 116-119 at y=11
pos[116] = (1, 11)
pos[117] = (5, 11)
pos[118] = (9, 11)
pos[119] = (13, 11)

# Main row 6: 120-135 at y=12
for i in range(16):
    pos[120 + i] = (i, 12)

# Flag qubits 136-139 at y=13

```

```

pos[136] = (3, 13)
pos[137] = (7, 13)
pos[138] = (11, 13)
pos[139] = (15, 13)

# Main row 7: 140-155 at y=14
for i in range(16):
    pos[140 + i] = (i, 14)

# Create figure with two subplots
fig, axes = plt.subplots(1, 2, figsize=(26, 16))

for ax_idx, (key, cfg) in enumerate(configs.items()):
    ax = axes[ax_idx]

    src = cfg['source_qubits']
    dst = cfg['dest_qubits']
    anc = cfg['ancilla_qubits']
    travel = cfg['travel_path']

    # Color nodes by role
    node_colors = {}
    for i in range(backend.num_qubits):
        if i in src:
            node_colors[i] = '#3498db' # Blue (Source)
        elif i in dst:
            node_colors[i] = '#e74c3c' # Red (Destination)
        elif i in anc:
            node_colors[i] = '#2ecc71' # Green (Syndrome Ancillas)
        elif i in travel:
            node_colors[i] = '#f1c40f' # Yellow (Travel Path)
        else:
            node_colors[i] = '#bdc3c7' # Light Gray

    # Draw edges
    for edge in edges:
        q0, q1 = edge[0], edge[1]
        if q0 in pos and q1 in pos:
            x0, y0 = pos[q0]
            x1, y1 = pos[q1]
            ax.plot([x0, x1], [y0, y1], 'cornflowerblue', linewidth=1.8, zorder=1)

    # Draw nodes
    node_radius = 0.38
    for i in range(backend.num_qubits):
        if i in pos:
            x, y = pos[i]
            circle = plt.Circle((x, y), node_radius, color=node_colors[i],
                                ec='black', linewidth=0.8, zorder=2)
            ax.add_patch(circle)
            ax.text(x, y, str(i), ha='center', va='center',
                    fontsize=6, fontweight='bold', zorder=3)

    ax.set_xlim(-1, 17)
    ax.set_ylim(-1, 16)
    ax.set_aspect('equal')
    ax.axis('off')

    # Calculate total SWAPs
    n = len(travel) + 1
    total_swaps = n + (n+1) + (n+2)

    ax.set_title(f"(3,1,1) Code Transport on {backend.name} ({cfg['name']})\n"
                 f"Source (Blue): {src[0]}-{src[1]}-{src[2]} | "
                 f"Destination (Red): {dst[0]}-{dst[1]}-{dst[2]} | "
                 f"Ancillas (Green): {anc[0]}, {anc[1]}\n"
                 f"Travel Path (Yellow): {travel} | Total SWAPs: {total_swaps}",
                 fontsize=11)

    # Legend
    legend_elements = [
        Patch(facecolor='#3498db', edgecolor='black', label=f'Source: {src}'),
        Patch(facecolor='#e74c3c', edgecolor='black', label=f'Destination: {dst}'),
        Patch(facecolor='#2ecc71', edgecolor='black', label=f'Syndrome Ancillas: {anc}'),
        Patch(facecolor='#f1c40f', edgecolor='black', label=f'Travel Path: {travel}'),
    ]
    ax.legend(handles=legend_elements, loc='upper right', fontsize=9)

plt.tight_layout()
plt.show()

# Print transport path details
print("\n=== Transport Path Summary ===")
for key, cfg in configs.items():
    src = cfg['source_qubits']
    dst = cfg['dest_qubits']
    travel = cfg['travel_path']
    n = len(travel) + 1

    full_path = [src[2]] + travel + [dst[2]]
    print(f"\n{cfg['name']}")
    print(f"Bottom: {src[2]} → {dst[2]}: {' → '.join(map(str, full_path))} ({n} SWAPs)")
    print(f>Data: {src[1]} → {dst[1]}: {n+1} SWAPs")

```

```
print(f" Top: {src[0]} → {dst[0]}: {n+2} SWAPs")
print(f" Total: {n + (n+1) + (n+2)} SWAPs")
```

FakeFez: 156 qubits, 352 edges

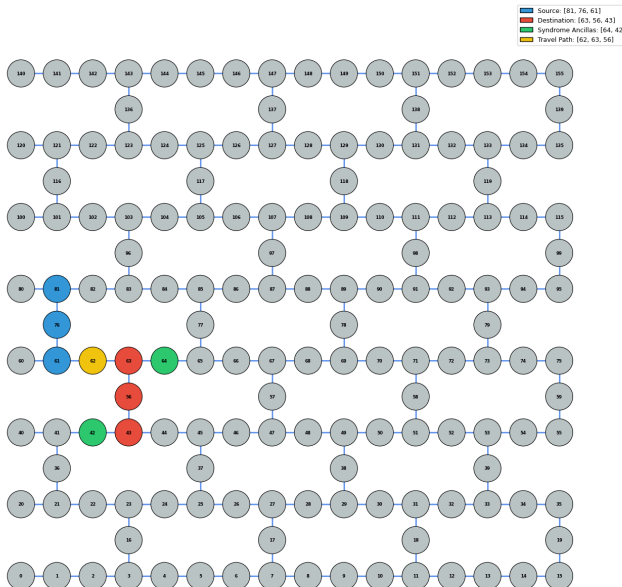
Vertical connections from row 0:

```
3 → [16]
7 → [17]
11 → [18]
15 → [19]
16 → [23]
17 → [27]
18 → [31]
19 → [35]
```

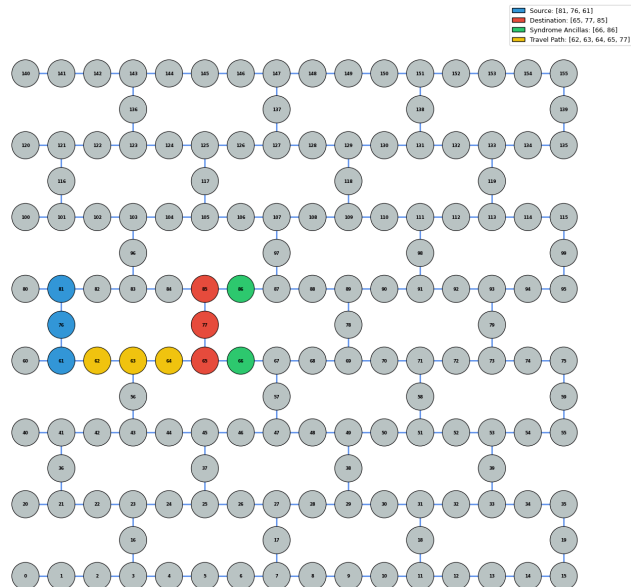
Checking connections for qubits 16–19:

```
16: [3, 23]
17: [7, 27]
18: [11, 31]
19: [15, 35]
```

(3,1,1) Code Transport on ibm_fez (Small Edge)
Source (Blue): 81-76-61 | Destination (Red): 63-56-43 | Ancillas (Green): 64, 42
Travel Path (Yellow): [62, 63, 56] | Total SWAPs: 15



(3,1,1) Code Transport on ibm_fez (Large Edge)
Source (Blue): 81-76-61 | Destination (Red): 65-77-85 | Ancillas (Green): 66, 86
Travel Path (Yellow): [62, 63, 64, 65, 77] | Total SWAPs: 21



=== Transport Path Summary ===

Small Edge:

```
Bottom: 61 → 43: 61 → 62 → 63 → 56 → 43 (4 SWAPs)
Data: 76 → 56: 5 SWAPs
Top: 81 → 63: 6 SWAPs
Total: 15 SWAPs
```

Large Edge:

```
Bottom: 61 → 85: 61 → 62 → 63 → 64 → 65 → 77 → 85 (6 SWAPs)
Data: 76 → 77: 7 SWAPs
Top: 81 → 65: 8 SWAPs
Total: 21 SWAPs
```

4. Dynamic SWAP Chain Generation

Define the generalized circuit builder that works with any configuration.

```
In [6]: def generate_swap_chains(source_qubits, dest_qubits, travel_path):
        """
        Generate SWAP chains for transporting [[3,1,1]] encoded qubits.

        The transport follows a "follow the leader" pattern:
        1. Bottom qubit moves first: src_bot → travel_path → dst_bot
        2. Data qubit follows: src_data → src_bot → travel_path → dst_bot → dst_data
        3. Top qubit follows: src_top → src_data → src_bot → travel_path → dst_bot → dst_data → dst_top
        """
        src_top, src_data, src_bot = source_qubits
        dst_top, dst_data, dst_bot = dest_qubits

        # Build full path from src_bot to dst_bot
        full_travel = [src_bot] + travel_path + [dst_bot]

        # SWAP chain for bottom qubit
        swap_bottom = [(full_travel[i], full_travel[i+1]) for i in range(len(full_travel)-1)]

        # SWAP chain for data qubit (follows bottom's path + one more)
        swap_data = [(src_data, src_bot)] + swap_bottom + [(dst_bot, dst_data)]

        # SWAP chain for top qubit (follows data's path + one more)
```

```

swap_top = [(src_top, src_data)] + swap_data + [(dst_data, dst_top)]

return swap_bottom, swap_data, swap_top

def build_311_transport_circuit(backend, config, initial_state='plus', verbose=False):
    """
    Build a [[3,1,1]] encoded qubit transport circuit with syndrome measurement.

    Parameters:
    -----
    backend : Backend
        The quantum backend (e.g., FakeFez)
    config : dict
        Configuration dictionary with source_qubits, dest_qubits, ancilla_qubits, travel_path
    initial_state : str
        Initial state of data qubit: 'zero', 'one', 'plus', or 'minus'
    verbose : bool
        Print circuit construction details

    Returns:
    -----
    QuantumCircuit
        The complete transport + syndrome circuit
    """
    source_qubits = config['source_qubits']
    dest_qubits = config['dest_qubits']
    ancilla_qubits = config['ancilla_qubits']
    travel_path = config['travel_path']

    src_top, src_data, src_bot = source_qubits
    dst_top, dst_data, dst_bot = dest_qubits
    anc_top, anc_bot = ancilla_qubits

    # Create circuit
    qc = QuantumCircuit(backend.num_qubits, 2)

    # =====
    # STEP 1: ENCODING
    # =====
    if initial_state == 'zero':
        pass
    elif initial_state == 'one':
        qc.x(src_data)
    elif initial_state == 'plus':
        qc.h(src_data)
    elif initial_state == 'minus':
        qc.x(src_data)
        qc.h(src_data)
    else:
        raise ValueError(f"Unknown initial_state: {initial_state}")

    qc.cx(src_data, src_top)
    qc.cx(src_data, src_bot)
    qc.barrier(label="Encode")

    # =====
    # STEP 2: TRANSPORT
    # =====
    swap_bottom, swap_data, swap_top = generate_swap_chains(
        source_qubits, dest_qubits, travel_path
    )

    for q1, q2 in swap_bottom:
        qc.swap(q1, q2)
    for q1, q2 in swap_data:
        qc.swap(q1, q2)
    for q1, q2 in swap_top:
        qc.swap(q1, q2)

    qc.barrier(label="Transport")

    if verbose:
        total_swaps = len(swap_bottom) + len(swap_data) + len(swap_top)
        print(f"{config['name']}: {total_swaps} SWAPS ({len(swap_bottom)}+{len(swap_data)}+{len(swap_top)})")

    # =====
    # STEP 3: SWAP-ASSISTED SYNDROME MEASUREMENT
    # =====
    # Syndrome 1: Z_top Z_data
    qc.cx(dst_top, anc_top)
    qc.swap(dst_data, dst_top)
    qc.cx(dst_top, anc_top)
    qc.measure(anc_top, 0)
    qc.swap(dst_top, dst_data)

    # Syndrome 2: Z_data Z_bot
    qc.cx(dst_bot, anc_bot)
    qc.swap(dst_bot, dst_data)
    qc.cx(dst_bot, anc_bot)
    qc.measure(anc_bot, 1)
    qc.swap(dst_data, dst_bot)

    qc.barrier(label="Syndrome")

```

```

    return qc

# Test the function
print("Circuit builder function defined: build_311_transport_circuit()")
print("\nSWAP chain calculation:")
for key, cfg in configs.items():
    swap_bot, swap_dat, swap_top = generate_swap_chains(
        cfg['source_qubits'], cfg['dest_qubits'], cfg['travel_path']
    )
    total = len(swap_bot) + len(swap_dat) + len(swap_top)
    print(f"  {cfg['name']}: {total} SWAPs ({len(swap_bot)}+{len(swap_dat)}+{len(swap_top)})")

```

Circuit builder function defined: build_311_transport_circuit()

SWAP chain calculation:

Small Edge: 18 SWAPs (4+6+8)
Large Edge: 24 SWAPs (6+8+10)

5. Build and Inspect Circuits

Build circuits for both paths and compare their properties.

```

In [7]: # Build circuits for both configurations
circuits = {}

print("=== Circuit Properties ===")
for key, cfg in configs.items():
    qc = build_311_transport_circuit(backend, cfg, initial_state='plus', verbose=True)
    circuits[key] = qc
    print(f"  Depth: {qc.depth()}, Gates: {qc.count_ops()}")
    print()

=== Circuit Properties ===
Small Edge: 18 SWAPs (4+6+8)
  Depth: 21, Gates: OrderedDict({'swap': 22, 'cx': 6, 'barrier': 3, 'measure': 2, 'h': 1})

Large Edge: 24 SWAPs (6+8+10)
  Depth: 23, Gates: OrderedDict({'swap': 28, 'cx': 6, 'barrier': 3, 'measure': 2, 'h': 1})

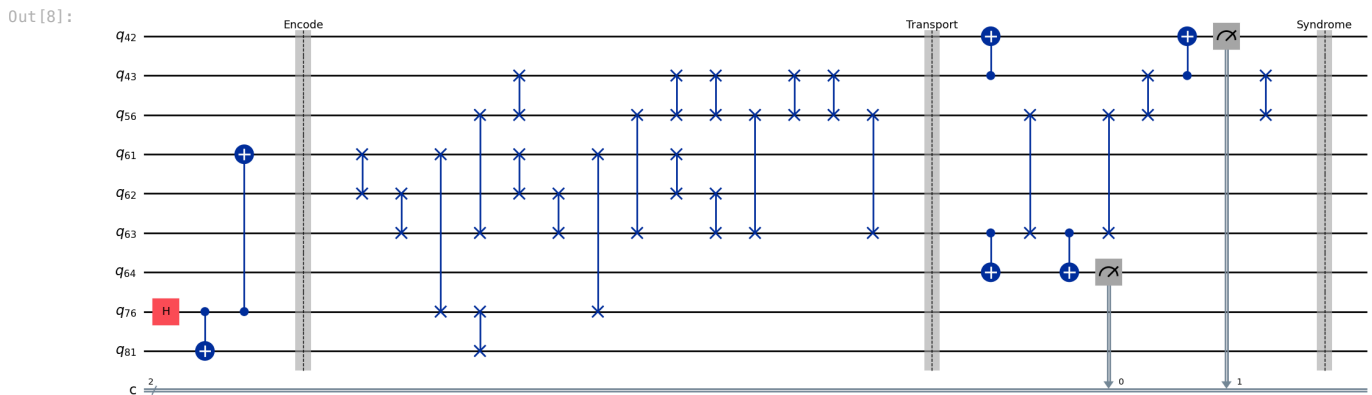
```

```

In [8]: # Draw the Small Edge circuit
print("Small Edge Circuit:")
circuits['small'].draw('mpl', fold=-1, idle_wires=False)

```

Small Edge Circuit:



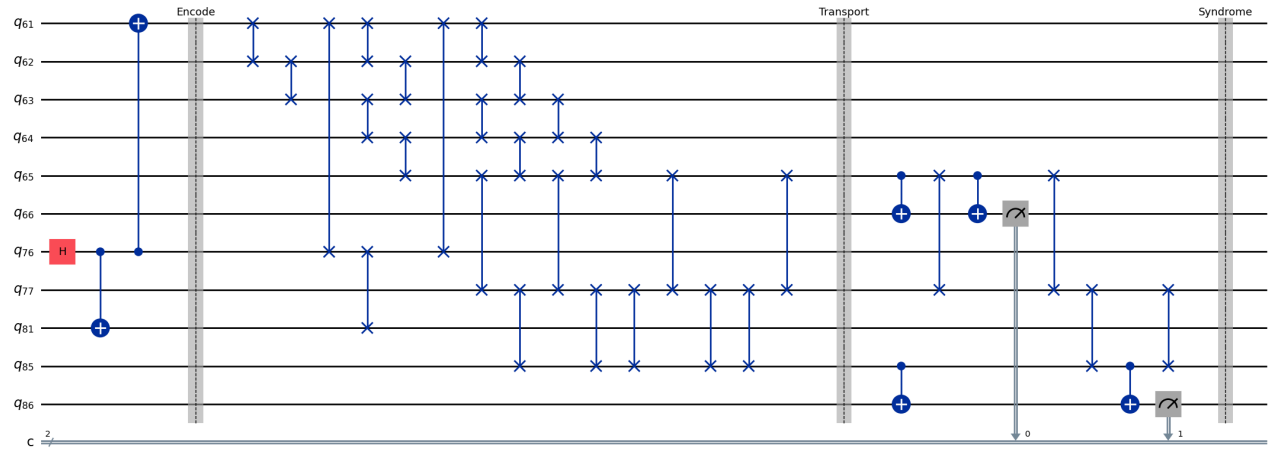
```

In [9]: # Draw the Large Edge circuit
print("Large Edge Circuit:")
circuits['large'].draw('mpl', fold=-1, idle_wires=False)

```

Large Edge Circuit:

Out[9]:



6. Run Experiments for All Initial States

Test both paths with all 4 initial states: $|0\rangle$, $|1\rangle$, $|+\rangle$, $|-\rangle$

```
In [10]: # Define test states
states_to_test = ['zero', 'one', 'plus', 'minus']
state_symbols = {'zero': '|0\rangle', 'one': '|1\rangle', 'plus': '|+\rangle', 'minus': '|-\rangle'}
encoded_states = {
    'zero': '|000\rangle',
    'one': '|111\rangle',
    'plus': '(|000\rangle+|111\rangle)/\sqrt{2}',
    'minus': '(|000\rangle-|111\rangle)/\sqrt{2}'
}

num_shots = 2048
all_results = {}

print("=" * 70)
print(f"Running [[3,1,1]] Transport Experiments on {backend.name}")
print(f"Mode: {'REAL HARDWARE' if USE_REAL_HARDWARE else 'SIMULATOR'}")
print("=" * 70)

# Set up execution based on backend type
if USE_REAL_HARDWARE:
    from qiskit_ibm_runtime import SamplerV2 as Sampler
    sampler = Sampler(backend)
    print("Using SamplerV2 for real hardware execution")

for path_key, cfg in configs.items():
    print(f"\n>>> {cfg['name']} Path <<<")
    print(f"    Source: {cfg['source_qubits']} -> Dest: {cfg['dest_qubits']}")

    all_results[path_key] = {}

    for state in states_to_test:
        print(f"\n    Testing {state_symbols[state]} -> {encoded_states[state]}...")

        # Build circuit
        qc = build_311_transport_circuit(backend, cfg, initial_state=state)

        # Transpile
        transpiled = transpile(
            qc, backend,
            initial_layout=list(range(backend.num_qubits)),
            optimization_level=0
        )

        # Run based on backend type
        if USE_REAL_HARDWARE:
            # Use SamplerV2 for real hardware
            job = sampler.run([transpiled], shots=num_shots)
            print(f"    Job ID: {job.job_id()}")
            result = job.result()
            # SamplerV2 returns data differently
            pub_result = result[0]
            counts = pub_result.data.c.get_counts()
        else:
            # Use backend.run() for simulator
            result = backend.run(transpiled, shots=num_shots).result()
            counts = result.get_counts()

        all_results[path_key][state] = counts

    success_rate = counts.get('00', 0) / num_shots * 100
    print(f"    Transpiled depth: {transpiled.depth()}")
    print(f"    Success rate (00): {success_rate:.1f}%")
```

```
print("\n" + "=" * 70)
print("All experiments complete!")
print("=" * 70)
```

```
=====
Running [[3,1,1]] Transport Experiments on ibm_fez
Mode: REAL HARDWARE
=====
```

```
Using SamplerV2 for real hardware execution
```

```
>>> Small Edge Path <<<
      Source: [81, 76, 61] → Dest: [63, 56, 43]
```

```
Testing |0⟩ → |000⟩...
Job ID: d5goit7ea9qs7391akmg
Transpiled depth: 104
Success rate (00): 73.6%
```

```
Testing |1⟩ → |111⟩...
Job ID: d5gojo2gim5s73ah3m60
Transpiled depth: 104
Success rate (00): 52.6%
```

```
Testing |+⟩ → (|000⟩+|111⟩)/√2...
Job ID: d5gojqfea9qs7391alng
Transpiled depth: 104
Success rate (00): 63.0%
```

```
Testing |-⟩ → (|000⟩-|111⟩)/√2...
Job ID: d5gojtf67pic7384houg
Transpiled depth: 105
Success rate (00): 63.7%
```

```
>>> Large Edge Path <<<
      Source: [81, 76, 61] → Dest: [65, 77, 85]
```

```
Testing |0⟩ → |000⟩...
Job ID: d5gojuv67pic7384hp0g
Transpiled depth: 116
Success rate (00): 59.2%
```

```
Testing |1⟩ → |111⟩...
Job ID: d5gok04pe0pc73algsu0
Transpiled depth: 116
Success rate (00): 50.9%
```

```
Testing |+⟩ → (|000⟩+|111⟩)/√2...
Job ID: d5gok1fea9qs7391am00
Transpiled depth: 116
Success rate (00): 55.0%
```

```
Testing |-⟩ → (|000⟩-|111⟩)/√2...
Job ID: d5gok2nea9qs7391am20
Transpiled depth: 117
Success rate (00): 56.5%
```

```
=====
All experiments complete!
=====
```

7. Analyze and Compare Results

Compare syndrome distributions between Small Edge and Large Edge paths.

```
In [11]: # Summary comparison table
print("=" * 80)
print("SYNDROME DISTRIBUTION COMPARISON")
print("=" * 80)

print(f"\n{'Path':<12} {'State':<8} {'Encoded':<22} {'00 (OK)':<10} {'01':<10} {'10':<10} {'11':<10}")
print("_" * 82)

summary_stats = {}

for path_key, cfg in configs.items():
    path_name = cfg['name']
    summary_stats[path_key] = []

    for state in states_to_test:
        counts = all_results[path_key][state]
        total = sum(counts.values())

        p00 = counts.get('00', 0) / total * 100
        p01 = counts.get('01', 0) / total * 100
        p10 = counts.get('10', 0) / total * 100
        p11 = counts.get('11', 0) / total * 100

        summary_stats[path_key].append(p00)

    print(f"{'path_name':<12} {'state_symbols[state]':<8} {'encoded_states[state]':<22} {p00:>6.1f}% {p01:>6.1f}% {p10:>6.1f}% {p11:>6.1f}%")

    print() # Blank line between paths
```

```

print("-" * 82)
print("\nAverage Success Rates:")
for path_key, cfg in configs.items():
    avg = np.mean(summary_stats[path_key])
    std = np.std(summary_stats[path_key])
    print(f"  {cfg['name']}: {avg:.1f}% ± {std:.1f}%")

```

SYNDROME DISTRIBUTION COMPARISON

Path	State	Encoded	00 (OK)	01	10	11
Small Edge	0⟩	000⟩	73.6%	8.4%	12.1%	6.0%
Small Edge	1⟩	111⟩	52.6%	16.1%	17.8%	13.5%
Small Edge	+⟩	(000⟩+ 111⟩)/√2	63.0%	13.0%	13.8%	10.3%
Small Edge	−⟩	(000⟩− 111⟩)/√2	63.7%	12.0%	14.3%	10.0%
Large Edge	0⟩	000⟩	59.2%	15.5%	14.6%	10.6%
Large Edge	1⟩	111⟩	50.9%	17.9%	16.6%	14.7%
Large Edge	+⟩	(000⟩+ 111⟩)/√2	55.0%	16.3%	15.0%	13.7%
Large Edge	−⟩	(000⟩− 111⟩)/√2	56.5%	15.6%	15.8%	12.1%

Average Success Rates:

Small Edge: 63.2% ± 7.4%
 Large Edge: 55.4% ± 3.0%

```

In [12]: # Visualize results
fig, axes = plt.subplots(2, 4, figsize=(16, 10))

syndrome_order = ['00', '01', '10', '11']
colors = ['#2ecc71', '#3498db', '#e74c3c', '#9b59b6']

for row_idx, (path_key, cfg) in enumerate(configs.items()):
    for col_idx, state in enumerate(states_to_test):
        ax = axes[row_idx, col_idx]
        counts = all_results[path_key][state]
        total = sum(counts.values())

        values = [counts.get(s, 0) for s in syndrome_order]
        percentages = [v / total * 100 for v in values]

        bars = ax.bar(syndrome_order, percentages, color=colors, edgecolor='black')

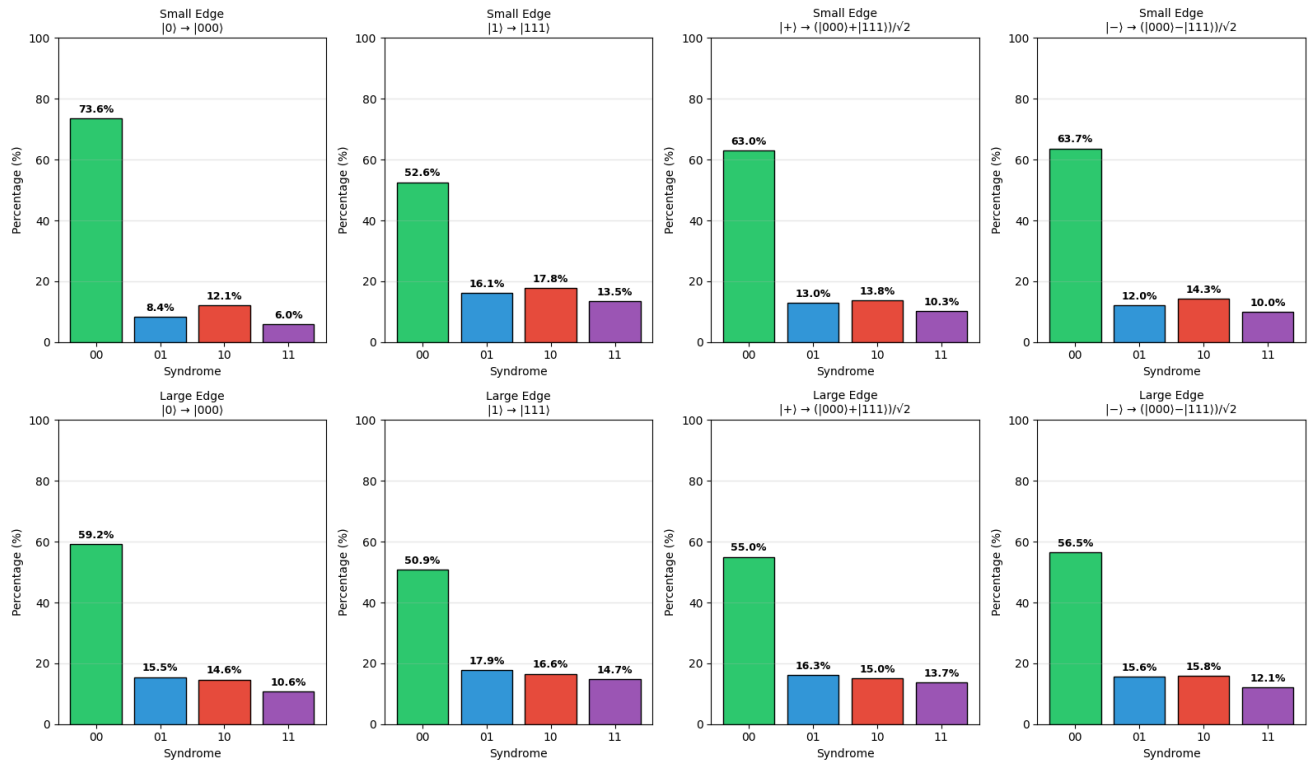
        for bar, pct in zip(bars, percentages):
            ax.annotate(f'{pct:.1f}%',
                        xy=(bar.get_x() + bar.get_width()/2, bar.get_height()),
                        xytext=(0, 3), textcoords='offset points',
                        ha='center', va='bottom', fontsize=9, fontweight='bold')

        ax.set_title(f'{cfg["name"]}\n{state_symbols[state]} → {encoded_states[state]}', fontsize=10)
        ax.set_xlabel('Syndrome')
        ax.set_ylabel('Percentage (%)')
        ax.set_ylim(0, 100)
        ax.grid(axis='y', alpha=0.3)

plt.suptitle(f'[[3,1,1]] Transport Results on {backend.name}\nSmall Edge (9 SWAPs) vs Large Edge (21 SWAPs)',
             fontsize=14, fontweight='bold', y=1.02)
plt.tight_layout()
plt.show()

```

[[3,1,1]] Transport Results on ibm_fez
Small Edge (9 SWAPs) vs Large Edge (21 SWAPs)



```
In [13]: # Bar chart comparing success rates by path
fig, ax = plt.subplots(figsize=(10, 6))

x = np.arange(len(states_to_test))
width = 0.35

small_rates = summary_stats['small']
large_rates = summary_stats['large']

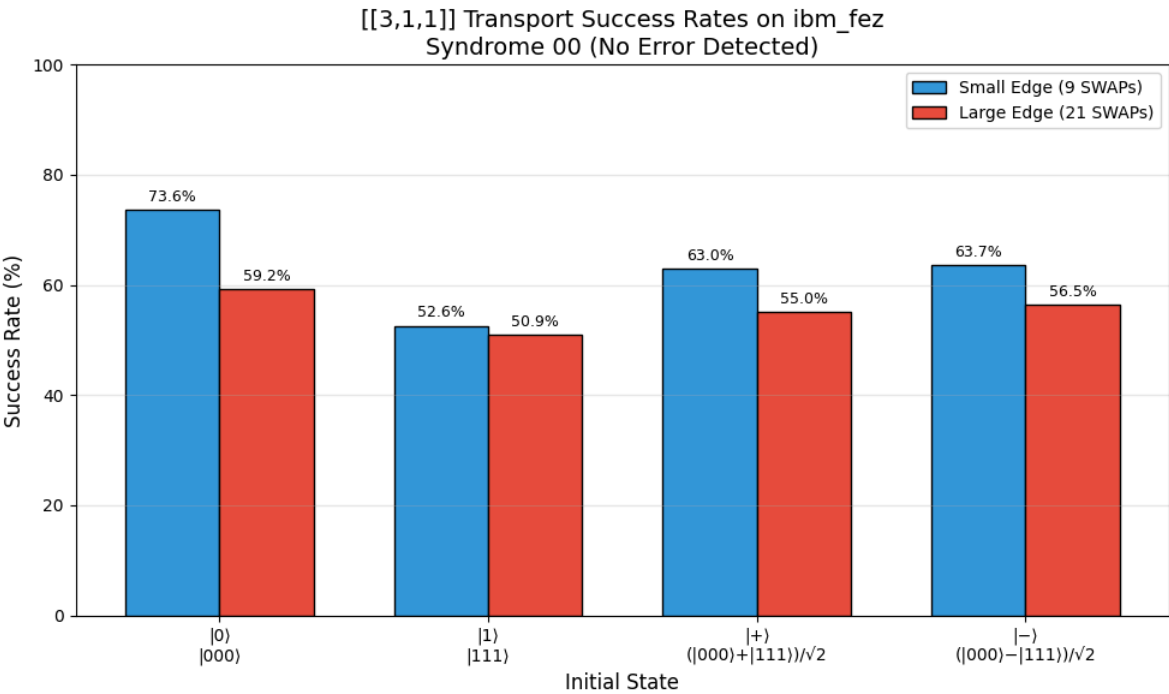
bars1 = ax.bar(x - width/2, small_rates, width, label='Small Edge (9 SWAPs)', color='#3498db', edgecolor='black')
bars2 = ax.bar(x + width/2, large_rates, width, label='Large Edge (21 SWAPs)', color='#e74c3c', edgecolor='black')

ax.set_xlabel('Initial State', fontsize=12)
ax.set_ylabel('Success Rate (%)', fontsize=12)
ax.set_title(f'[[3,1,1]] Transport Success Rates on {backend.name}\nSyndrome 00 (No Error Detected)', fontsize=14)
ax.set_xticks(x)
ax.set_xticklabels([f'{state_symbols[s]}\n{encoded_states[s]} for s in states_to_test'])
ax.legend()
ax.set_ylim(0, 100)
ax.grid(axis='y', alpha=0.3)

# Add value labels
for bars in [bars1, bars2]:
    for bar in bars:
        height = bar.get_height()
        ax.annotate(f'{height:.1f}%',
                    xy=(bar.get_x() + bar.get_width()/2, height),
                    xytext=(0, 3), textcoords='offset points',
                    ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()

# Print summary
print(f"\n=== Summary ===")
print(f"Small Edge avg: {np.mean(small_rates):.1f}%")
print(f"Large Edge avg: {np.mean(large_rates):.1f}%")
print(f"Difference: {np.mean(small_rates) - np.mean(large_rates):.1f}% (Small Edge is better)")
```



=== Summary ===
Small Edge avg: 63.2%
Large Edge avg: 55.4%
Difference: 7.8% (Small Edge is better)

8. Conclusions

Key Findings

- 1. **Small Edge (9 SWAPs)** has higher success rates than **Large Edge (21 SWAPs)**
- 2. Error rates scale with transport distance (number of SWAPs)
- 3. All initial states show similar syndrome distributions (as expected for [[3,1,1]] code)

Circuit Complexity Comparison

Path	Transport SWAPs	Estimated CX Gates
Small Edge	9	~27 + syndrome
Large Edge	21	~63 + syndrome

Syndrome Interpretation

Syndrome	Meaning
00	No error detected
01	Error on top qubit
10	Error on bottom qubit
11	Error on middle/data qubit